

物联网安全课程实验报告

实验三



实验名称： 物联网设备加密通信设计与实现 ”

姓名： 邢清画

小组： 邢清画，田晋宇，肖文翰

学号： 2212039

专业： 物联网工程

提交日期： 2024. 10. 20

一、实验目的

了解目前主流的基于云的物联网通信原理，学会使用基本的密码学工具，并在消费物联网应用场景下构建安全加密通信方案。

二、实验要求及要点

回答问题：

1. 智能家居设备的使用与工业控制系统面临风险有何差异？
2. 列举几种可用来实现加密通信的常见密码算法，并进行对比分析。
3. 什么是虚拟机的 NAT 模式、桥接模式、Host-only 模式？

要点：

- 实验目标
- 组员分工情况（须有一位组员专门从攻击者视角做安全评估）
- 方案设计
- 方案实现（包括网络拓扑，实现技术细节，功能效果演示，等等）
- 系统方案安全性评估（包括系统设计的不足和未来改进思路等）
- 每位成员的收获与感悟，体会“红蓝”对抗对构建安全系统的作用
- 提交源代码

三、实验内容

1. 基于 MQTT 协议，编程模拟实现智能家居应用场景下用户控制物联网设备的原型程序。原型系统至少包含如下功能：

(1) 支持至少 2 个用户，至少 3 个物联网设备（空调、灯泡、插座不同类型）同时在线

注：物联网设备硬件功能仅用代码模拟即可，如输出“on”表示灯打开

(2) 在不同的实体计算机（或虚拟机、手机）上部署 MQTT 服务器与 MQTT 客户端

(3) 利用密码学知识与工具，查询相关资料，使该系统能抵御指令重放攻击。

(4) 模拟核心的智能家居功能：用户操控设备、查看设备状态、设备绑定

2. 从攻击者的视角对本组的原型系统实现开展安全评估

提示：请考虑不同的威胁模型，即不同的应用场景与敌手能力。例如，攻击者可能合法购买一个同款设备逆向分析，甚至可能共享使用过目标设备。

四、实验过程

4.1 安装 emqx

MQTT 消息服务器中的 EMQ X 下载地址：

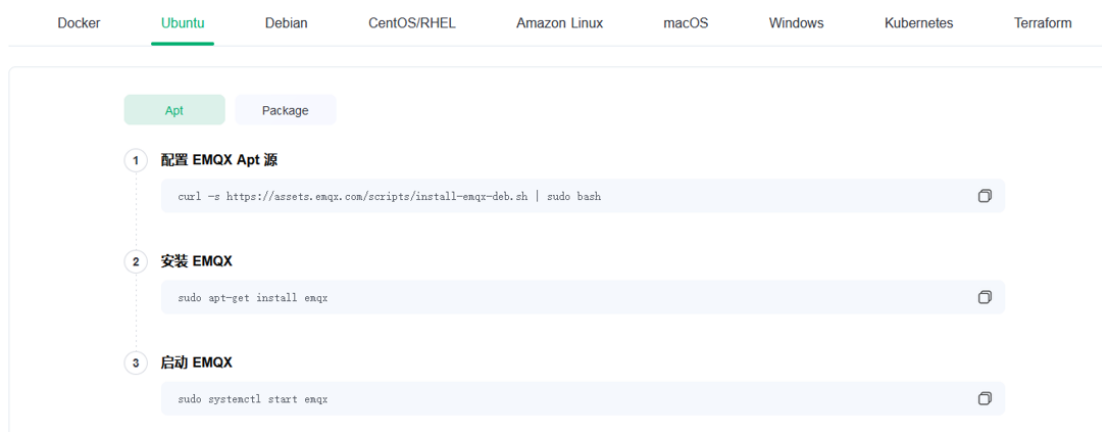
<https://www.emqx.io/zh/downloads?os=Ubuntu>

配置 EMQX Apt 源：`curl -s https://assets.emqx.com/scripts/install-emqx-deb.sh | sudo bash`

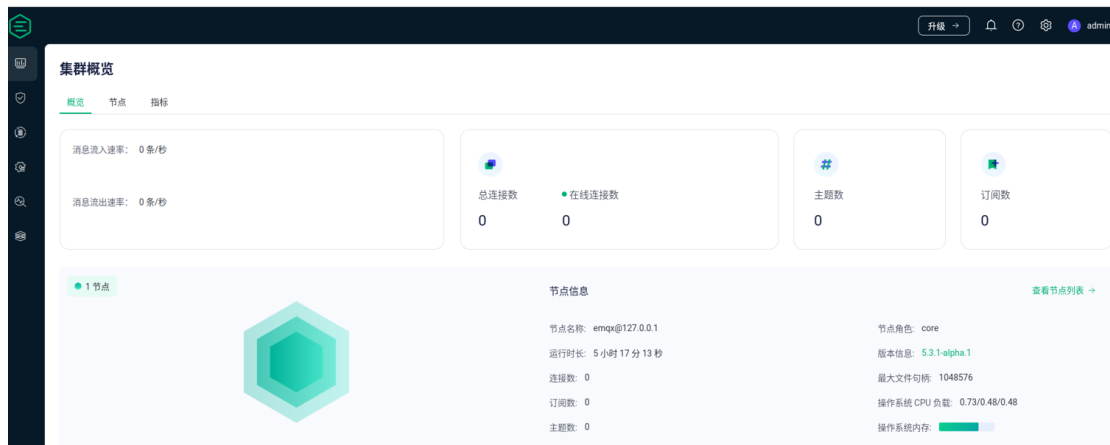
安装 EMQX：`sudo apt-get install emqx`

启动 EMQX：`sudo systemctl start emqx`

下载 EMQX



安装完成后访问本机即可打开页面

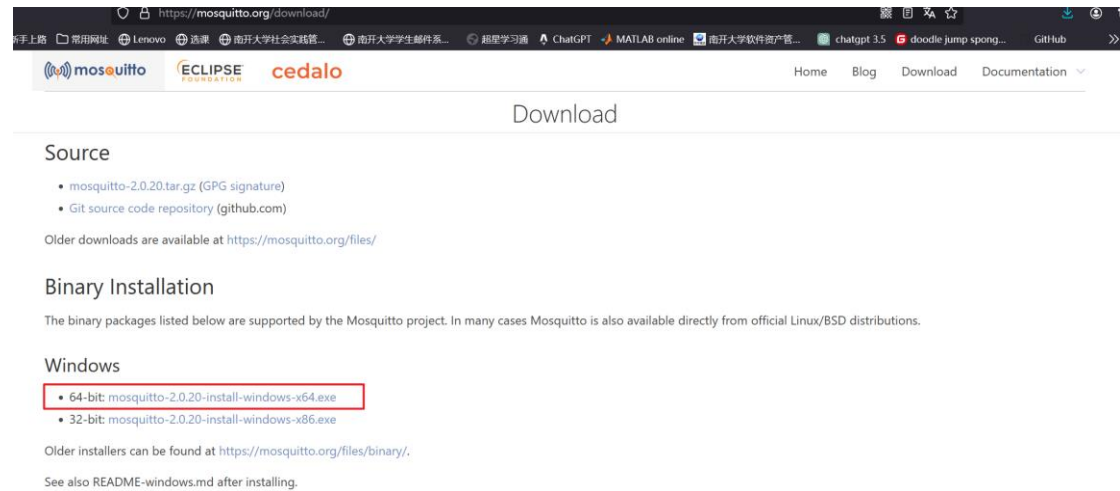


Ps: 后来选择了在 Windows 下进行实验（后续实验报告内容），上面是进行的尝试。

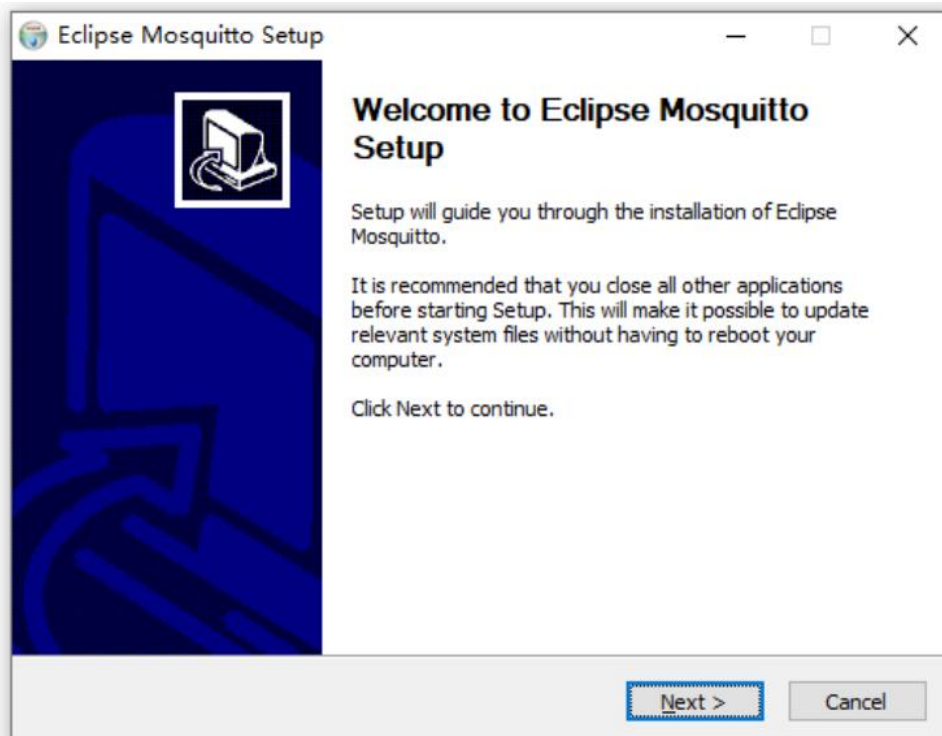
4.2 使用 mosquitto 进行测试

4.2.1 mosquitto 安装

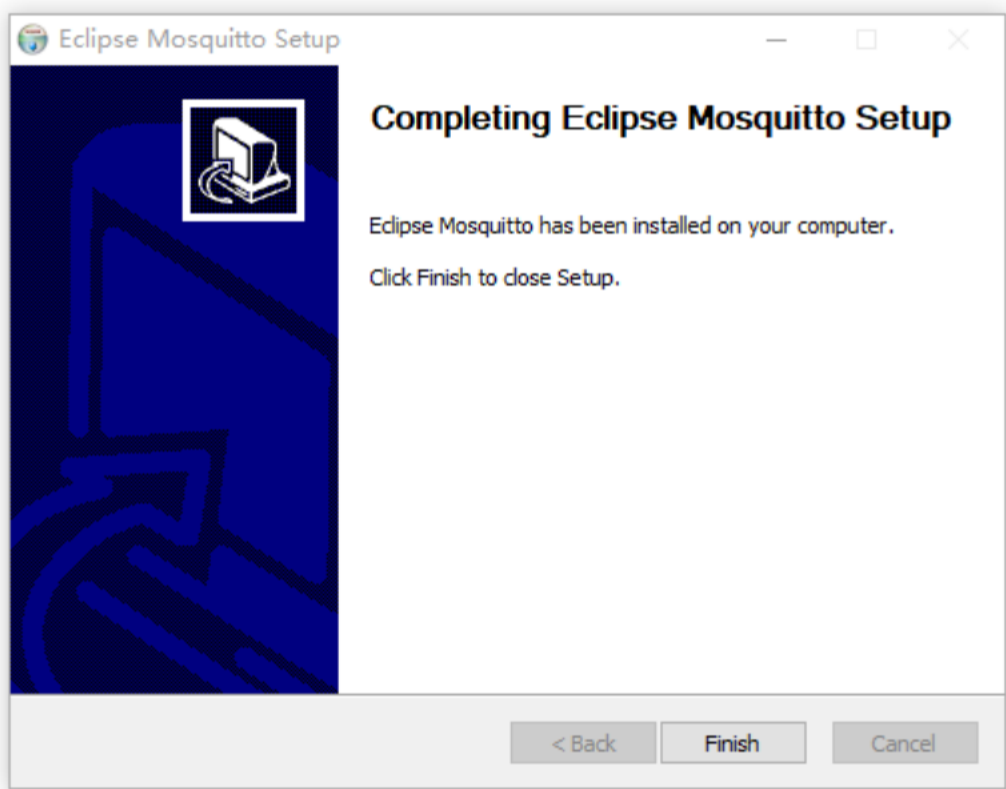
进入 mosquitto 下载网址: <https://mosquitto.org/download/>



右键点击 mosquitto-2.0.20-install-windows-x64.exe 以管理员身份进行安装:



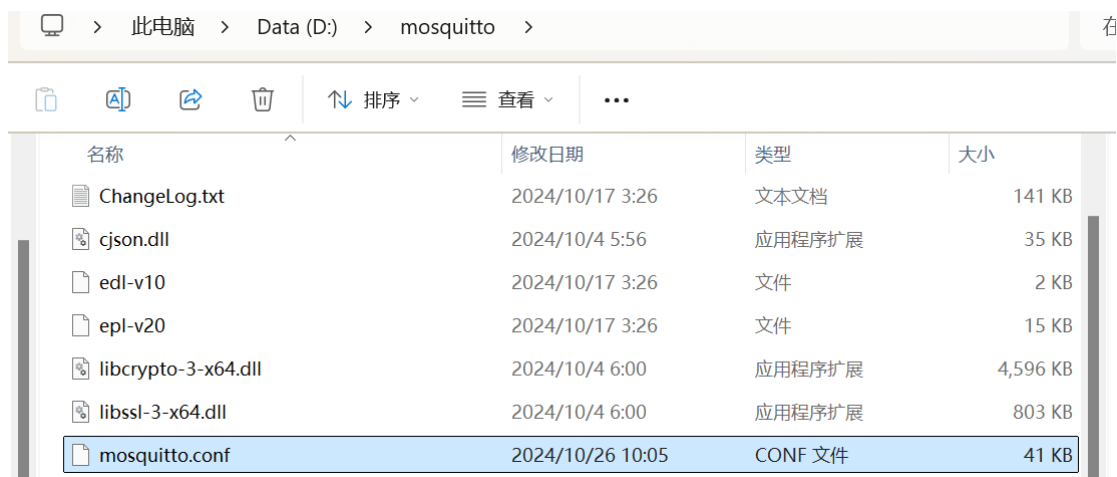
不断点击 Next>直到安装完毕:



点击“Finish”按钮，至此安装完成。

4.2.2 修改配置文件并启动 mosquitto 服务

1. 配置文件位置：D:\mosquitto\mosquitto.conf，如下：



2、修改侦听端口和开启用户验证（即设置用户名和密码）

2.1 修改侦听端口

打开 mosquitto.conf，找到 listener，取消前面的#注释，并指定要侦听的端口，如下：

```
235 # listener port-number [ip address/host name/unix socket path]
236 listener 7788/10.136.188.246
```

2.2 设置用户名密码

2.2.1 找到 allow_anonymous 节点，这个节点作用是，是否开启匿名用户登录，

默认是 true。打开此项配置（将前面的 # 号去掉）之后将其值改为 false:

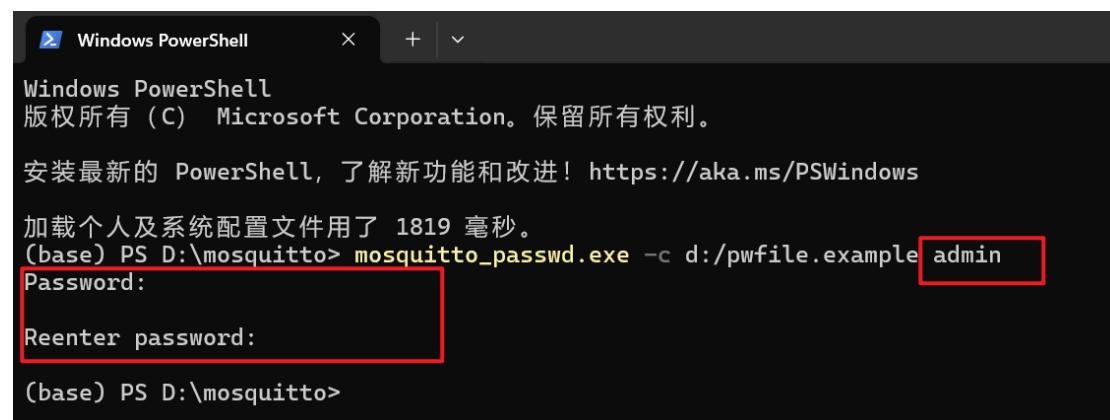
```
23 # allow_anonymous 修改前
24 allow_anonymous false 修改后
```

2.2.2 找到 password_file 节点, 这个节点是告诉服务器你要配置的用户将存放在哪里。打开此配置并指定 pwfile.example 文件路径（注意是绝对路径）

```
27
28 password_file D:\mosquitto\pwfile.example
```

3、在管理工具->服务, 中找到 Mosquitto Broker, 启动 mosquitto 服务。

4、以管理员身份打开 CMD, 并切换到 d:/mosquitto 目录下, 运行以下命令添加用户名和密码:



```
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

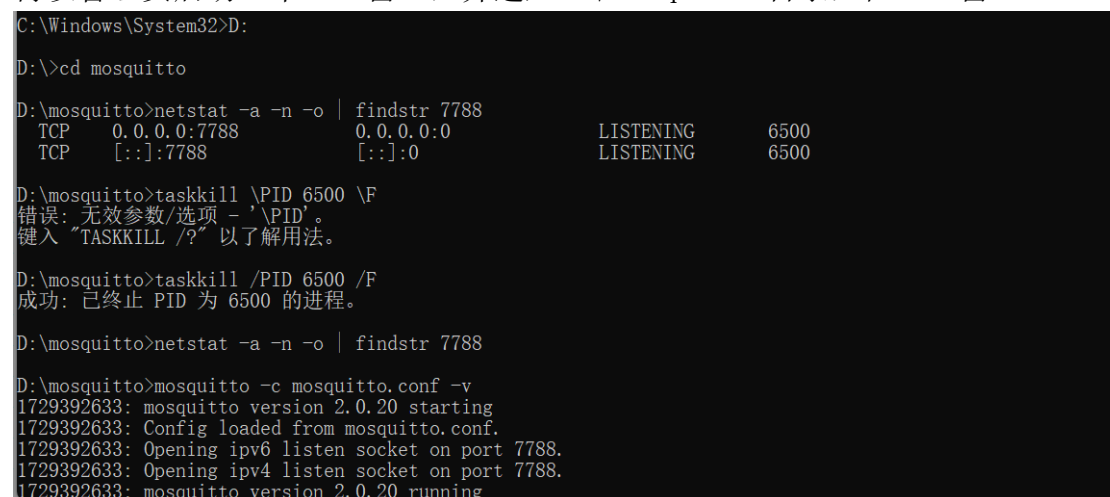
安装最新的 PowerShell, 了解新功能和改进! https://aka.ms/PSWindows

加载个人及系统配置文件用了 1819 毫秒。
(base) PS D:\mosquitto> mosquitto_passwd.exe -c d:/pwfile.example admin
Password:
Reenter password:
(base) PS D:\mosquitto>
```

检查 pwfile.example 文件, 查看是否添加成功。可以看到 SHE256 加密后的密码。

4.2.3 利用 mosquitto 工具测试订阅与发布

再以管理员启动 3 个 CMD 窗口, 并进入 d:/mosquitto 目录, 在 cmd 窗口 1:



```
C:\Windows\System32>D:
D:\>cd mosquitto
D:\mosquitto>netstat -a -n -o | findstr 7788
TCP    0.0.0.0:7788      0.0.0.0:0        LISTENING        6500
TCP    [::]:7788        [::]:0           LISTENING        6500
D:\mosquitto>taskkill \PID 6500 /F
错误: 无效参数/选项 - '\PID'。
键入 "TASKKILL /?" 以了解用法。
D:\mosquitto>taskkill /PID 6500 /F
成功: 已终止 PID 为 6500 的进程。
D:\mosquitto>netstat -a -n -o | findstr 7788
D:\mosquitto>mosquitto -c mosquitto.conf -v
1729392633: mosquitto version 2.0.20 starting
1729392633: Config loaded from mosquitto.conf.
1729392633: Opening ipv6 listen socket on port 7788.
1729392633: Opening ipv4 listen socket on port 7788.
1729392633: mosquitto version 2.0.20 running
```

cmd 窗口 2 作为订阅者, cmd 窗口 3 作为发布者。

在 cmd 窗口 1 中执行以下命令, 进行消息订阅:

mosquitto_sub.exe -h 127.0.0.1 -p 7788 -u admin -P 123456 -v -t sensor

在 cmd 窗口 2 中执行以下命令, 进行消息发送:

mosquitto_pub.exe -h 127.0.0.1 -p 7788 -u admin -P 123456 -t sensor -m "消息测试"

```

PS D:\mosquitto> .\mosquitto_sub.exe -h 127.0.0.1 -p 7788 -u admin -P 123456 -v -t sensor
sensor 消息测试
sensor xwhtest

```

```

命令提示符
Microsoft Windows [版本 10.0.22631.4317]
(c) Microsoft Corporation。保留所有权利。

C:\Users\xwh>D:

D:\>cd mosquitto

D:\mosquitto>.\mosquitto_pub.exe -h 127.0.0.1 -p 7788 -u admin -P 123456 -t sensor -m "消息测试"

D:\mosquitto>.\mosquitto_pub.exe -h 127.0.0.1 -p 7788 -u admin -P 123456 -t sensor -m "xwhtest"

D:\mosquitto>

```

可以看到连接初步建设成功

pip install paho-mqtt==1.6.1

```

Project
  实验三 C:\Users\xwh\Desktop\物联网安全实验\实验三
    client1.py
    client2.py
    mqtt-v3.1.1标准文档.pdf
    MQTT开发相关资源链接.txt
    server.py
    《深入浅出密码学——常用加密技术原理与应用》.pdf
    密码编码学与网络安全——原理与实践（第七版）.pdf
    计算机网络安全课件.pptx
  External Libraries
  Scratches and Consoles

server.py
1  import time
2  import paho.mqtt.client as mqtt
3
4  # 创建 MQTT 客户端实例
5  client = mqtt.Client(client_id="publisher")
6  client.username_pw_set(username="admin", password="123456")
7
8  client.connect(host="192.168.137.1", port=7788, keepalive=60)
9  client.loop_start()
10
11 # 发布消息到指定主题
12 for i in range(10):
13     client.publish(topic="sensor", payload=f"Test message {i}")
14     #client.publish("sensor", f"Test message {i}")
15     time.sleep(1) # 给客户端足够时间接收
16
17 # 断开连接前等待一会儿，确保消息发送完毕
18 time.sleep(5)
19
20 client.loop_stop() # 停止循环
21 client.disconnect()
22
Run
server (1) x
D:\Anaconda\python.exe C:\Users\xwh\Desktop\物联网安全实验\实验三\server.py
Process finished with exit code 0

```

Project

- code C:\tjy\code
 - client1.py
 - client2.py
 - server.py
- External Libraries
- Scratches and Consoles

client2.py

```
1 import paho.mqtt.client as mqtt
2
3 # 当客户端收到连接响应时的回调函数
4 1 usage
5 def on_connect(client, userdata, flags, rc):
6     if rc == 0:
7         print("Connected successfully.")
8         client.subscribe("sensor")
9     else:
10        print(f"Failed to connect, return code {rc}")
11
12 # 当收到消息时的回调函数
13 1 usage
14 def on_message(client, userdata, msg):
15     print(f"在{msg.topic}主题下收到: {msg.payload.decode()}")
16     if ("2" in msg.payload.decode()):
17         print("开机")
```

Run client2

```
D:\Miniconda3\python.exe C:\tjy\code\client2.py
C:\tjy\code\client2.py:18: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="2")
Connected successfully.
在sensor主题下收到: Test message 0
在sensor主题下收到: Test message 1
开机
在sensor主题下收到: Test message 2
在sensor主题下收到: Test message 3
在sensor主题下收到: Test message 4
在sensor主题下收到: Test message 5
在sensor主题下收到: Test message 6
在sensor主题下收到: Test message 7
在sensor主题下收到: Test message 8
```

Project

- 实验三 C:\Xing\大三上\物联网安全\实验三
 - code
 - .idea
 - client1.py
 - client2.py
 - server.py
 - 2211999_那清画_物联网安全实验报告3.docx
 - ~\$11999_那清画_物联网安全实验报告3.docx
 - 物联网安全实验报告3.docx
- External Libraries
- Scratches and Consoles

client1.py

```
1 import paho.mqtt.client as mqtt
2
3 # 当客户端收到连接响应时的回调函数
4 1 usage
5 def on_connect(client, userdata, flags, rc):
6     if rc == 0:
7         print("Connected successfully.")
8         client.subscribe("sensor")
9     else:
10        print(f"Failed to connect, return code {rc}")
11
12 # 当收到消息时的回调函数
13 1 usage
14 def on_message(client, userdata, msg):
15     print(f"在{msg.topic}主题下收到: {msg.payload.decode()}")
16     if ("1" in msg.payload.decode()):
17         print("开机")
```

Run client1

```
D:\Miniconda3\python.exe C:\Xing\大三上\物联网安全\实验三\code\client1.py
C:\Xing\大三上\物联网安全\实验三\code\client1.py:18: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="1")
Connected successfully.
在sensor主题下收到: Test message 0
在sensor主题下收到: Test message 1
开机
在sensor主题下收到: Test message 2
在sensor主题下收到: Test message 3
在sensor主题下收到: Test message 4
在sensor主题下收到: Test message 5
在sensor主题下收到: Test message 6
在sensor主题下收到: Test message 7
在sensor主题下收到: Test message 8
```



```
D:\Miniconda3\python.exe C:\Xing\大三上\物联网安全\实验三\code\client1.py
C:\Xing\大三上\物联网安全\实验三\code\client1.py:18: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="1")
Connected successfully.
在sensor主题下收到: Test message 0
在sensor主题下收到: Test message 1
开机
在sensor主题下收到: Test message 2
在sensor主题下收到: Test message 3
在sensor主题下收到: Test message 4
在sensor主题下收到: Test message 5
在sensor主题下收到: Test message 6
在sensor主题下收到: Test message 7
在sensor主题下收到: Test message 8
在sensor主题下收到: Test message 9
在sensor主题下收到: Test message 0
在sensor主题下收到: Test message 0
在sensor主题下收到: Test message 1
```

```
Connected successfully.
Connected successfully.
Connected successfully.
Connected successfully.
在sensor主题下收到: Test message 1
开机
Connected successfully.
在sensor主题下收到: Test message 3
Connected successfully.
在sensor主题下收到: Test message 5
Connected successfully.
在sensor主题下收到: Test message 7
Connected successfully.
在sensor主题下收到: Test message 9
Connected successfully.
Connected successfully.
Connected successfully.
Connected successfully.
Connected successfully.
Connected successfully.
Connected successfully.
```

上述实验在小组三个人的不同实体计算机上部署 MQTT 服务器与 MQTT 客户端。通过连接同一热点，验证在三台电脑上消息互通成功（通过终端红色指示框中的信息区分三台设备）

下面我们将继续编程模拟实现智能家居应用场景下用户控制物联网设备的原型程序。

4.3 系统对重放攻击的安全措施

在这部分中，我们基于 MQTT 协议实现了一个多用户支持的智能家居控制系统，重点探讨了消息传递的安全性和用户设备状态的同步。系统包含多个用户，每个用户可以控制和查询多个智能设备的状态，例如灯光、加热器和音响。同时，实

验引入了防止指令重放攻击的安全措施，以确保系统的安全性和消息的有效性。

程序功能概述：

1. **设备绑定：**用户在第一次使用设备前需要发送绑定请求。只有绑定成功的用户才能控制或查询设备的状态。服务器在收到绑定请求后，建立用户与设备的绑定关系。
2. **设备控制：**用户发送控制指令（如打开或关闭设备），服务器验证该用户是否绑定了设备，只有绑定成功的用户才能操作设备。设备状态更改后会广播给所有用户。
3. **设备状态查询：**用户可以查询设备当前的状态。查询响应仅返回给请求的用户，不影响其他用户。
4. **安全机制：**引入时间戳、随机数（Nonce）和数字签名，以防止重放攻击并确保消息的完整性和真实性。

程序结构与工作流程

1. 服务器端程序

服务器端程序主要负责：

- **管理设备状态：**维护所有设备的全局状态，并接收来自客户端的控制和查询请求。
- **设备绑定管理：**服务器端维护一个用户-设备绑定关系字典，只有绑定了设备的用户才可以控制或查询设备。
- **安全验证：**
 - 在接收控制和查询请求时，服务器验证请求中的时间戳和随机数（Nonce），确保消息唯一性。
 - 使用 RSA 数字签名验证请求来源的真实性，防止恶意伪造的请求。

服务器端的主要流程：

- 接收到绑定请求时，检查用户是否已绑定该设备，若未绑定则记录绑定关系，并返回绑定成功的响应。
- 控制或查询请求处理前，检查用户是否绑定了该设备，若未绑定则拒绝操作；若已绑定则执行控制或查询。
- 控制指令成功执行后，设备状态更新会广播给所有用户，确保多用户的状态同步。

2. 客户端程序

客户端程序主要负责：

- **设备绑定：**用户可以通过发送绑定请求，将设备与自身账户绑定。
- **发送控制指令：**用户在成功绑定设备后，可以发送控制指令（如“打开灯”），服务器处理后将最新状态广播给所有用户。
- **状态查询：**用户可以查询设备的当前状态，服务器仅返回查询请求用户的设备状态。
- **安全性实现：**每个请求附带时间戳和随机数，并对消息内容进行数字签名。服务器验证签名，以确保请求来源可信。

客户端的主要流程：

- 连接到服务器后，订阅私有的响应主题和广播主题，接收与自身相关的设备更新。
- 根据用户输入判断是发送绑定请求、控制指令还是状态查询请求，按需与服务器通信。
- 绑定请求和查询请求的响应仅显示给当前用户，而控制指令的响应会通过广播主题发送给所有用户。

首先展示实验效果：

```

C:\WINDOWS\system32\cmd
命令提示符: 输入 'light', 'heater' 或 'speaker' 查看状态
输入命令: light
命令已发送: light
light 当前状态: on,1738879273.9681234
输入命令: light 已 打开
light 当前状态: on,1738879362.4319735
lightoff
命令已发送: lightoff
lightoff 已 关闭
输入命令: light
命令已发送: light
light 当前状态: off,1738879376.5218762
Traceback (most recent call last):
  File "F:\Desktop\Grade3\物联网安全\safety_lab\client1.py", line 89, in <module>
    send_command(user_input)
    *****
KeyboardInterrupt
C:\WINDOWS\system32\cmd
(network) F:\Desktop\Grade3\物联网安全\safety_lab\python client1.py
F:\Desktop\Grade3\物联网安全\safety_lab\client1.py:32: DeprecationWarning: Callback API version 1 is deprecated, update
to latest version
client = mqtt.Client()
输入以下指令以控制设备:
按任意键以继续设备:
灯光: 输入 'lighton' 开启, 输入 'lightoff' 关闭
加热器: 输入 'heateron' 开启, 输入 'heateroff' 关闭
扬声器: 输入 'speakeron' 开启, 输入 'speakeroff' 关闭
查看设备状态: 输入 'light', 'heater' 或 'speaker' 查看状态
输入命令:

C:\WINDOWS\system32\cmd
[2020-11-06 15:49:31] [INFO] 已启动并初始化: lightoff 已 关闭
*****
[2020-11-06 15:49:36] [DEBUG] 控制设备 - 用户ID: user1 | 主题: devices/user1/control | 内容: light,1738879376.5218762,
1e6f90e721e0e0f49b1ab0f3eef78e399b1125e6d6d4c6f77d3e6e0330ec777d6c6d6d6d2c6b89b1f8e85f4e6d299e1c19165
af09802c48f7eeefc6e2f8ac2a69e8e3cb812772c6726e6b0f321a6a681126a22c7f6d6:1752c909f9e2c6871ef8b1ba6b15f22b6f6e6f
9a2d607268207cc93b6f6e2c9b6e6b9e2d2a6f6d6e1d1b5392a3b6e6b0f61a6b1ab122766367b8786c2b6e6b1e26e277e6c65
f6e61195a33a6c3f6e6b0b1169e6b0f6c3a6562a6e17a3f4b6b676b6e6c2e9fa13c6e6b0e6c9af72b1f6b6c6e6b6f611a6b1b6e6c7b6
819b6e6137a62343a6e64e233dc6d6e6993e6b6b03f6e69899
*****
[2020-11-06 15:49:36] [INFO] 设备验证成功
*****
[2020-11-06 15:49:36] [INFO] 已发送 user1 的 light 状态: off
*****
Traceback (most recent call last):
  File "F:\Desktop\Grade3\物联网安全\safety_lab\server.py", line 137, in <module>
    while True:
      KeyboardInterrupt
C:\WINDOWS\system32\cmd
(network) F:\Desktop\Grade3\物联网安全\safety_lab\python server.py
F:\Desktop\Grade3\物联网安全\safety_lab\server.py:40: DeprecationWarning: Callback API version 1 is deprecated, update
to latest version
client = mqtt.Client()
[2020-11-06 15:49:45] [INFO] 连接并启动MQTT服务器成功
*****

C:\WINDOWS\system32\cmd
1738879273: Received PUBLISH from auto-EDA973F-24D3-5096-38E2-38E2B8A8D08C (00, q0, r0, m0, 'devices/user1/control', ...
(553 bytes))
1738879273: Sending PUBLISH to auto-69217A8D-80F6-5C18-FE6A-F6D471F32BCE (00, q0, r0, m0, 'devices/user1/control', ...
(553 bytes))
1738879273: Received PUBLISH from auto-69217A8D-80F6-5C18-FE6A-F6D471F32BCE (00, q0, r0, m0, 'devices/user1/response', ...
(52 bytes))
1738879273: Sending PUBLISH to auto-1A3A4D84-A2FC-0974-F98B-E92385311178 (00, q0, r0, m0, 'devices/user1/response', ...
(52 bytes))
1738879273: Sending PUBLISH to auto-EDA973F-24D3-5096-38E2-38E2B8A8D08C (00, q0, r0, m0, 'devices/user1/response', ...
(52 bytes))
1738879273: Received PINGREQ from auto-69217A8D-80F6-5C18-FE6A-F6D471F32BCE
1738879273: Sending PINGRESP to auto-69217A8D-80F6-5C18-FE6A-F6D471F32BCE
1738879273: Received PINGREQ from auto-1A3A4D84-A2FC-0974-F98B-E92385311178
1738879273: Sending PINGRESP to auto-1A3A4D84-A2FC-0974-F98B-E92385311178
1738879273: Received PINGREQ from auto-EDA973F-24D3-5096-38E2-38E2B8A8D08C
1738879273: Sending PINGRESP to auto-EDA973F-24D3-5096-38E2-38E2B8A8D08C
1738879273: Received PINGREQ from auto-69217A8D-80F6-5C18-FE6A-F6D471F32BCE
1738879273: Sending PINGRESP to auto-69217A8D-80F6-5C18-FE6A-F6D471F32BCE
1738879273: Received PUBLISH from auto-1A3A4D84-A2FC-0974-F98B-E92385311178 (00, q0, r0, m0, 'devices/user1/control', ...
(553 bytes))
1738879273: Sending PUBLISH to auto-69217A8D-80F6-5C18-FE6A-F6D471F32BCE (00, q0, r0, m0, 'devices/user1/control', ...
(553 bytes))
1738879273: Received PUBLISH from auto-69217A8D-80F6-5C18-FE6A-F6D471F32BCE (00, q0, r0, m0, 'devices/broadcast', ...
(16 bytes))
1738879273: Sending PUBLISH to auto-1A3A4D84-A2FC-0974-F98B-E92385311178 (00, q0, r0, m0, 'devices/broadcast', ...
(16 bytes))
1738879273: Sending PUBLISH to auto-EDA973F-24D3-5096-38E2-38E2B8A8D08C (00, q0, r0, m0, 'devices/broadcast', ...
(16 bytes))
1738879273: Received PUBLISH from auto-1A3A4D84-A2FC-0974-F98B-E92385311178 (00, q0, r0, m0, 'devices/user1/control', ...
(553 bytes))

```

在一个终端窗口输入指令 `mosquitto -c mosquitto.conf -v` 启动 mosquitto 服务。

一个终端窗口启动预先编写好的服务器端 python 程序，另外两个窗口开启 userid 分别为 user1 和 user2 的客户端窗口，我们可以验证多用户对设备控制时是否会同步更新。

客户端需要先绑定设备才能对设备进行一系列操作，否则服务器端将拒绝访问请求：

```
C:\WINDOWS\system32\cmd. X + v

self._sock = self._create_socket()
File "F:\python_env\anaconda3\envs\network\Lib\site-packages\paho\mqtt\client.py", line 4609, in _create_socket
sock = self._create_socket_connection()
File "F:\python_env\anaconda3\envs\network\Lib\site-packages\paho\mqtt\client.py", line 4640, in _create_socket_connection
return socket.create_connection(addr, timeout=self._connect_timeout, source_address=source)
File "F:\python_env\anaconda3\envs\network\Lib\socket.py", line 863, in create_connection
raise exceptions[0]
File "F:\python_env\anaconda3\envs\network\Lib\socket.py", line 848, in create_connection
sock.connect(sa)
ConnectionRefusedError: [WinError 10061] 由于目标计算机积极拒绝，无法连接。

(network) F:\Desktop\Grade3\物联网安全\safety_lab3>python client1.py
F:\Desktop\Grade3\物联网安全\safety_lab3\client1.py:33: DeprecationWarning: Callback API version 1 is deprecated, update
to latest version
client = mqtt.Client()
输入以下指令以控制设备：

绑定设备：输入 'bind <设备名称>'，例如 'bind light'
连接到MQTT服务器成功
控制设备：输入 'lighton' 开启灯，输入 'lightoff' 关闭灯
加热器：输入 'heateron' 开启，输入 'heateroff' 关闭
音响：输入 'speakeron' 开启，输入 'speakeroff' 关闭
查询设备状态：输入 'light'、'heater' 或 'speaker' 查询状态
请输入指令：light
未绑定设备 light，请先绑定
请输入指令：
```

我们输入 bind light 指令绑定之后，在 user1 客户端窗口输入指令 light，查询灯的状态，可以获取灯的状态：

```
绑定设备：输入 'bind <设备名称>'，例如 'bind light'
连接到MQTT服务器成功
控制设备：输入 'lighton' 开启灯，输入 'lightoff' 关闭灯
加热器：输入 'heateron' 开启，输入 'heateroff' 关闭
音响：输入 'speakeron' 开启，输入 'speakeroff' 关闭
查询设备状态：输入 'light'、'heater' 或 'speaker' 查询状态
请输入指令：light
未绑定设备 light，请先绑定
请输入指令：bind light
绑定设备 light 成功
请输入指令：light
light 当前状态：off
请输入指令：
```

此时我们查看服务器端的调试信息，发现消息内容是一大段加密后的密文，加密是通过 RSA 数字签名技术实现的，后面进行具体介绍。

```
=====[2024-11-06 16:09:13] [INFO] 连接到MQTT服务器成功=====

[2024-11-06 16:09:20] [DEBUG] 接收到消息 - 用户ID: user1 | 主题: devices/user1/control | 内容: light,1730880560.5888557,HZ3jC0pyUP11FfXW,6b2ca3205c00b2fdb34359f2d3f02abc5ccd03e25533918152bfa10e87db787e2bb0e9dba1732ffadde074fd0cdde323c8e5629a24fe9d8293dc4c42b1e4ba5b467190dc6989195835aa9ca0e6cce4f165abdca4a87b18cbe168677fc70135962e9dbd7c1ebd5b165ca4bf4c7f2257a060f5f5ceebc70c7cd2b5e7da6f4ab23abc6bb54be771d5a69ad8c41e3dff4a9988f0a9ac01dc212408af3dad837a78651a502ccbe97cd8ad70506f468ec197785dbd04435546135f00e924488a61fe2988c36eb9bdb248a2bd2c7a44e4b1b605ba835aa587788004acbae6ef3830fbae007281e822e129e1d77d0a4cc5115d1226e505ee4320c188e5df7645a8afe57

[2024-11-06 16:09:20] [WARNING] 用户 user1 未绑定设备 light，操作拒绝

[2024-11-06 16:10:48] [DEBUG] 接收到消息 - 用户ID: user1 | 主题: devices/user1/bind | 内容: light

[2024-11-06 16:10:48] [INFO] 用户 user1 成功绑定设备 light

[2024-11-06 16:10:53] [DEBUG] 接收到消息 - 用户ID: user1 | 主题: devices/user1/control | 内容: light,1730880653.451347,VVpbRIItait818WSQ,b09f2c18077f1ab286cb739c2775a1ccc6b4646d4793f462953d05d4f63fcd7318471a521b45627e2a2a575659c1debd01d6449706f9be1f870f83194383563cab1681620baf5582aa82b2d60932d0071d623d501b01cafdd66b7d45f5f8f4b6947413994c028559c00e6f4b4695a4302efc51c83c20d2b33cad0a2b737a7138f6efd1157fe3c06072accdd2b26ec6f921fbad39fff4c6d57aa7ef50283c371d81901f2787406fe5d704ed7711abc50909aff9bb26e761a47b2636372fa83e8894f72a99fb98461996a77a3eca0ecaeb586591c8172689d2588d42f33cb7fd1dec9c36d7f87f69c9620d2765a5f44989e93760b024b45fa8030ee9b260f821fd17c
```

同样的我们为 user2 绑定灯，输入 lighton 指令可以在终端中看到灯已经打开，同时灯的状态会广播到 user1：

```
(network) F:\Desktop\Grade3\物联网安全\safety_lab3>python client2.py
F:\Desktop\Grade3\物联网安全\safety_lab3\client2.py:33: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client()
输入以下指令以控制设备:
连接到MQTT服务器成功

绑定设备: 输入 'bind <设备名称>', 例如 'bind light'
控制设备: 输入 'lighton' 开启灯, 输入 'lightoff' 关闭灯
加热器: 输入 'heateron' 开启, 输入 'heateroff' 关闭
音响: 输入 'speakeron' 开启, 输入 'speakeroff' 关闭
查询设备状态: 输入 'light'、'heater' 或 'speaker' 查询状态
请输入指令: bind light
绑定设备 light 成功
请输入指令: lighton
light 已 打开
请输入指令:

(network) F:\Desktop\Grade3\物联网安全\safety_lab3>python client1.py
F:\Desktop\Grade3\物联网安全\safety_lab3\client1.py:33: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client()
输入以下指令以控制设备:

绑定设备: 输入 'bind <设备名称>', 例如 'bind light'
连接到MQTT服务器成功
控制设备: 输入 'lighton' 开启灯, 输入 'lightoff' 关闭灯
加热器: 输入 'heateron' 开启, 输入 'heateroff' 关闭
音响: 输入 'speakeron' 开启, 输入 'speakeroff' 关闭
查询设备状态: 输入 'light'、'heater' 或 'speaker' 查询状态
请输入指令: light
未绑定设备 light, 请先绑定
请输入指令: bind light
绑定设备 light 成功
请输入指令: light
light 当前状态: off
请输入指令: light 已 打开
```

我们还实现了对多设备的控制，热水器和音响，以下是对热水器控制的示例：

```
(network) F:\Desktop\Grade3\物联网安全\safety_lab3>python client2.py
F:\Desktop\Grade3\物联网安全\safety_lab3\client2.py:33: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client()
输入以下指令以控制设备:

连接到MQTT服务器成功
绑定设备: 输入 'bind <设备名称>', 例如 'bind light'
控制设备: 输入 'lighton' 开启灯, 输入 'lightoff' 关闭灯
加热器: 输入 'heateron' 开启, 输入 'heateroff' 关闭
音响: 输入 'speakeron' 开启, 输入 'speakeroff' 关闭
查询设备状态: 输入 'light'、'heater' 或 'speaker' 查询状态
请输入指令: bind heater
绑定设备 heater 成功
请输入指令: heater
heater 当前状态: off
请输入指令: heateron
heater 已 打开
请输入指令: heater
heater 当前状态: on
请输入指令:
```

接下来对重点对重放攻击的安全措施进行介绍：

1. 时间戳验证

作用：时间戳（timestamp）用于记录每条指令的发送时间，服务器通过验证时间戳与当前时间的差值，确保指令的及时性，从而有效防止旧指令的重放。

实现：

在客户端的 send_command 函数中，生成时间戳并添加到指令消息中：

```
def send_command(command):
    timestamp = str(time.time())
    nonce = generate_nonce() # 生成随机数
    message = f"{command},{timestamp},{nonce}".encode()

    # 使用私钥签名消息
    signature = private_key.sign(
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

    # 将签名转换为可传输的字符串格式
    signature_hex = signature.hex()

    # 构造包含签名的消息
    command_with_signature = f"{command},{timestamp},{nonce},{signature_hex}"
```

服务器端在 `process_control_or_query` 函数中，接收指令并提取时间戳，通过对比时间戳和服务器当前时间判断消息的有效性。例如，可以拒绝时间超过一定范围（如 60 秒）的消息。

```
# 时间戳验证
current_time = time.time()
if current_time - float(timestamp) > 60:
    log_info(message=f"指令已超时，操作拒绝", level="WARNING")
    client.publish(topic=f"devices/{user_id}/response", payload="指令已超时，操作拒绝")
    return
```

通过时间戳验证，系统可以确保只有在特定时间范围内发送的指令才会被服务器执行，防止旧指令被重放利用。

2. 随机数 (Nonce)

作用：随机数 (Nonce) 为每条指令生成唯一的标识符，即使内容相同的指令也会有不同的随机数，从而防止同一指令的重放。

实现：

在客户端的 `send_command` 函数中，使用 `generate_nonce` 函数生成唯一的随机数，将其添加到指令消息中：

```
def send_command(command):
    timestamp = str(time.time())
    nonce = generate_nonce() # 生成随机数
    message = f"{command},{timestamp},{nonce}".encode()

    # 使用私钥签名消息
    signature = private_key.sign(
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

    # 将签名转换为可传输的字符串格式
    signature_hex = signature.hex()

    # 构造包含签名的消息
    command_with_signature = f"{command},{timestamp},{nonce},{signature_hex}"
```

服务器端在 `process_control_or_query` 函数中，提取并检查随机数。如果随机数之前已经处理过，则视为重放攻击并拒绝执行指令。服务器维护一个 `processed_nonces` 集合，记录已处理的随机数：

```
# 随机数 (Nonce) 验证
if user_id not in processed_nonces:
    processed_nonces[user_id] = set()
if nonce in processed_nonces[user_id]:
    log_info(message=f"检测到重放攻击 - 用户ID: {user_id} | Nonce: {nonce} 已处理", level="WARNING")
    client.publish(topic=f"devices/{user_id}/response", payload="检测到重放攻击，操作拒绝")
    return
else:
    processed_nonces[user_id].add(nonce)
```

通过随机数的唯一性，即使攻击者捕获了用户的指令，也无法重放指令，因为服务器会检测到重复的随机数并拒绝执行。

3. RSA 数字签名

RSA 签名是一种非对称加密算法，使用一对公私钥进行加密和解密。私钥用于签名消息，公钥用于验证签名。

RSA 数字签名的工作原理

签名过程：

1. 消息内容通过哈希函数生成一个哈希值。
2. 使用私钥对哈希值进行加密，生成数字签名。
3. 将原始消息和签名一起发送。

验证过程：

4. 接收端使用公钥对数字签名进行解密，恢复哈希值。
5. 对收到的原始消息进行哈希处理，得到新的哈希值。
6. 比较两个哈希值是否一致，若一致，则验证消息的完整性和来源。

实现：

首先我们生成一对公钥和私钥：

```
1 from cryptography.hazmat.primitives.asymmetric import rsa
2 from cryptography.hazmat.primitives import serialization
3 from cryptography.hazmat.backends import default_backend
4
5 # 生成私钥和公钥
6 private_key = rsa.generate_private_key(
7     public_exponent=65537,
8     key_size=2048,
9     backend=default_backend()
10 )
11
12 # 保存私钥
13 with open("private_key.pem", "wb") as f:
14     f.write(private_key.private_bytes(
15         encoding=serialization.Encoding.PEM,
16         format=serialization.PrivateFormat.TraditionalOpenSSL,
17         encryption_algorithm=serialization.NoEncryption()
18     ))
19
20 # 保存公钥
21 public_key = private_key.public_key()
22 with open("public_key.pem", "wb") as f:
23     f.write(public_key.public_bytes(
24         encoding=serialization.Encoding.PEM,
25         format=serialization.PublicFormat.SubjectPublicKeyInfo
26     ))
27
28 print("RSA 密钥对已生成并保存")
29
```

客户端在 `send_command` 函数中对完整的指令消息进行 RSA 签名，将签名添加到指令中：


```

message = f"{command},{timestamp},{nonce}".encode()

# 使用私钥签名消息
signature = private_key.sign(
    message,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

# 将签名转换为可传输的字符串格式
signature_hex = signature.hex()

# 构造包含签名的消息
command_with_signature = f"{command},{timestamp},{nonce},{signature_hex}"

```

服务器端在 `process_control_or_query` 函数中，接收到指令后，使用公钥对消息的签名进行验证。如果签名无效，则拒绝执行指令：

```

# 验证签名
message = f"{command},{timestamp},{nonce}".encode()
signature = bytes.fromhex(signature_hex)
try:
    public_key.verify(
        signature,
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
    log_info("签名验证通过")
except Exception as e:
    log_info(message=f"签名验证失败: {e}", level="ERROR")
    client.publish(topic=f"devices/{user_id}/response", payload="签名验证失败，操作拒绝")
    return

```

通过数字签名，系统确保只有经过合法用户签名的指令才会被服务器执行，从而有效防止恶意用户重放或伪造指令。

从攻击者的视角对本组的原型系统实现开展安全评估

答：从攻击者的视角评估本组的原型系统，可以识别出潜在的攻击方式，并分析系统的防御机制和可能的改进之处。下面是几种典型的攻击方法及其应对情况：

1. 重放攻击

攻击目标：攻击者可能截获某个用户的指令后，尝试重放相同指令（如再次打开或关闭设备），企图控制设备或干扰系统正常运行。

系统防御机制：

- **时间戳验证：**系统在指令中附加了时间戳，允许的时间窗口限制在 60 秒以内。任何超出该时间窗口的指令都将被拒绝处理。
- **随机数（Nonce）验证：**每条指令都包含一个唯一的随机数，服务器端维护了已处理随机数的集合。如果随机数已经存在，系统将拒绝执行该指令。

安全评估：

该系统的时间戳和随机数机制提供了有效的防重放保护，攻击者无法重放截获的旧指令。即使攻击者获取了指令内容，指令中包含的时间戳和随机数都无法被再利用。

攻击改进与建议：

攻击者若想绕过该防御机制，必须能够即时生成有效的的时间戳和随机数，并具备合法的签名。这基本上要求攻击者拥有合法的私钥，难度较高。

2. 伪造指令

攻击目标：攻击者试图伪造指令，向服务器发送假冒的控制或查询请求，以获得未经授权的设备控制权。

系统防御机制：

- **RSA 签名验证：**每条指令在发送前都使用 RSA 签名。服务器端会验证签名的真实性，确保指令来源可靠。只有合法的用户私钥签名才能通过验证。

安全评估：

RSA 签名提供了良好的防伪保护，攻击者如果不拥有私钥，将无法伪造有效的指令并通过服务器端的签名验证。

攻击改进与建议：

除非攻击者获取用户私钥，伪造攻击基本无法实施。为进一步提高安全性，可以考虑使用双因素身份验证或加强私钥保护措施，防止私钥被泄露。

3. 中间人攻击

攻击目标：攻击者充当用户与服务器之间的中间人，尝试拦截、篡改或伪造通信内容，达到误导服务器或用户的目的。

系统防御机制：

- **签名验证：**即使攻击者拦截通信内容，也无法对其篡改，因为篡改会导致签名验证失败。
- **时间戳和随机数：**中间人即便尝试延迟传输内容或重发指令，时间戳和随机数验证也会使重放指令失效。

安全评估：

签名和时间戳随机数机制可以有效抵御中间人攻击，确保指令的完整性与不可篡改性。

攻击改进与建议：

若攻击者尝试进行复杂的中间人攻击，可以考虑在通信中使用 SSL/TLS 加密，以防止攻击者在拦截内容的同时还解密传输内容。

4. 拒绝服务攻击

攻击目标：攻击者向服务器发送大量无效请求，意图耗尽服务器资源，使系统无法响应正常请求。

系统防御机制：

当前系统的设计中没有特别的防御机制来抵御 DoS 攻击。

安全评估：

在大规模无效请求或伪造指令的情况下，服务器可能会陷入资源耗尽状态，导致系统无法响应合法用户的请求。

攻击改进与建议：

可以在服务器端实现限流和黑名单机制，监测并限制每个用户的请求频率。此外，可以添加异常检测，自动屏蔽可能的恶意 IP。

5. 未授权访问（绕过绑定关系）

攻击目标：攻击者试图绕过设备绑定关系，控制未授权的设备或查询其状态。

系统防御机制：

- **设备绑定验证：**在执行控制或查询操作之前，系统会验证用户是否已绑定该设备。只有绑定设备的用户才可进行控制和查询操作。

安全评估：

设备绑定机制确保了只有获得授权的用户才能控制和查询设备。该设计可以有效防止未授权访问。

攻击改进与建议：

攻击者可能尝试利用系统漏洞绕过绑定验证。例如，发送绑定设备的伪造请求。因此，系统应严格检查每条绑定请求的签名真实性，以防止绑定关系被恶意篡改。

总体安全评估

我们实现的系统通过**时间戳、随机数、RSA 签名验证、设备绑定**等多重安全机制有效抵御了常见的网络攻击手段。它在以下几个方面具有较高的安全性：

1. **防止重放攻击：**通过时间戳和随机数机制有效防止指令重放。
2. **防伪造攻击：**RSA 签名确保了每条指令的真实性和完整性。
3. **防篡改和中间人攻击：**签名机制保证了指令内容的不可篡改性。
4. **访问控制：**通过设备绑定关系，系统确保了用户对设备的控制权限。

改进方向

1. **SSL/TLS 加密：**增加传输层加密，可以进一步增强系统在公开网络上的抗中间人攻击能力。
2. **频率限制与 IP 黑名单：**通过限流和黑名单机制，提高系统的抗拒绝服务攻击能力。
3. **日志和审计：**加强系统日志记录与审计，可以更早发现和分析潜在的恶意攻击行为。

五、回答问题

1. 智能家居设备的使用与工业控制系统面临风险有何差异？

答：

智能家居设备的风险主要集中在个人隐私和家庭安全上。这些设备通常连接在家庭 Wi-Fi 网络中，安全防护措施相对较弱，容易受到攻击。由于成本限制和用户安全意识不足，许多设备缺乏有效的安全更新机制，默认密码也容易被破解。因此，攻击者可能通过摄像头、门锁等设备入侵，导致隐私泄露、财产损失或设备失控，但影响通常限于单个家庭或个体。

相比之下，工业控制系统的安全风险更为严重，因为它们负责关键基础设施的运行，如电网、工厂和交通系统。工业系统面临的攻击动机更复杂，可能涉及经济利益或政治目的，其攻击后果往往影响更广泛，包括生产中断、环境污染，甚至威胁公共安全。此外，工业系统的安全防护更为严格，通常有多层防护措施并需符合严格的合规标准，但由于系统老化和复杂性增加，仍然面临被高级攻击威胁的风险。

工业控制系统通常使用专业的通信协议，如 MODBUS、DNP3 等，以确保数据的完整性和安全性。智能家居设备通常使用更常见的通信协议，如 Wi-Fi、蓝牙等，通常没有专门的安全保障。因此工业控制系统面临的风险主要是防止对基础设施和生产流程造成破坏，导致生产中断和安全风险。智能家居设备的风险是对个人隐私和数据泄露构成威胁。

2. 列举几种可用来实现加密通信的常见密码算法，并进行对比分析。

答：

1. 对称加密算法

对称加密算法是一种在加密和解密过程中使用相同密钥的加密方式，具有高效快速的特点，适合于大规模数据的加密。对称加密算法主要用于保护数据传输的内容。

特点：

- **速度快：**对称加密计算量相对较小，因此加密和解密的速度都很快，适合于实时数据传输。
- **密钥管理困难：**由于加密和解密使用同一密钥，因此需要确保通信双方都安全地获得该密钥。
- **适用场景：**常用于保护传输层（如 TLS）、VPN 等需要快速和高效加密的场景。

常见算法：

- **AES (Advanced Encryption Standard)：**AES 是最常用的对称加密算法，支持 128、192、256 位密钥长度，提供强大的加密强度且计算高效，适合在各种网络通信和存储加密场景中使用。
- **ChaCha20：**这是一种流加密算法，具有 256 位密钥，专为低功耗和资源受限的环境设计，广泛用于现代移动应用中的安全数据传输，特别是在 TLS 和 VPN 协议中应用广泛。
- **3DES (Triple DES)：**3DES 是对旧的 DES 算法的增强，通过三重加密来提高

安全性。然而，由于计算速度慢，逐渐被 AES 等算法取代，主要在旧系统中用于兼容性。

2. 非对称加密算法

非对称加密算法使用一对密钥——公钥和私钥，其中公钥用于加密，私钥用于解密。非对称加密算法通常用于密钥交换和身份验证，以确保安全通信。

特点：

- **密钥管理简单：**公钥可以公开共享，私钥则由拥有者保管，适合实现公开的加密通信。
- **计算开销大：**非对称加密的运算量较大，速度较慢，通常不直接用于大数据量的加密。
- **适用场景：**用于密钥交换、数字签名、身份认证等，如 TLS 中进行对称密钥交换、VPN 建立安全连接。

常见算法：

- **RSA (Rivest-Shamir-Adleman)：**RSA 是一种非常普遍的非对称加密算法，通常使用 2048 位或更高的密钥长度，广泛用于安全通信中的密钥交换和数字签名，如在 HTTPS 中确保连接的安全性。
- **ECC (Elliptic Curve Cryptography)：**ECC 是一种基于椭圆曲线的加密算法，相对于 RSA 提供相同安全级别时，密钥长度更短，计算效率更高，特别适合用于资源有限的设备和移动应用。

3. 杂凑算法（哈希算法）

杂凑算法用于将任意长度的输入数据映射为固定长度的哈希值或“摘要”，不直接用于加密数据，而是用于数据完整性验证和数字签名。杂凑算法确保数据在传输或存储过程中未被篡改。

特点：

- **不可逆：**杂凑算法是单向的，即由哈希值无法反向推导出原始数据。
- **抗碰撞性：**不同的输入应生成唯一的哈希值，避免产生相同的哈希值（碰撞）。
- **适用场景：**杂凑算法用于数字签名、数据完整性验证等，在加密通信中保证数据的完整性和可靠性。

常见算法：

- **SHA-256 (Secure Hash Algorithm 256-bit)：**SHA-256 是一种流行的杂凑算法，生成 256 位的哈希值，广泛应用于数字签名和区块链等需要数据完整性保护的场景。
- **MD5 (Message Digest Algorithm 5)：**MD5 生成 128 位哈希值，曾广泛用于数据完整性校验，但由于安全性不足，现在逐渐被 SHA-256 等更安全的算法替代。

3. 什么是虚拟机的 NAT 模式、桥接模式、Host-only 模式？

答：(1) NAT 模式：在 NAT 模式下，虚拟机的网络流量通过主机的网络连接进行转发。虚拟机使用主机的 IP 地址来访问外部网络，但外部网络无法直接访问虚拟机。

(2) 桥接模式：在桥接模式下，虚拟机可以直接连接到宿主机所在的物理网络，就像是与物理网络中的另一台计算机连接一样。虚拟机会获得与物理网络中其他设备相同的网络访问权限，并被分配一个与宿主机在同一网络中的独立 IP

地址。这使得虚拟机能够直接与外部网络进行通信。

(3)Host-only 模式：在 Host-only 模式下，虚拟机可以与宿主机进行通信，但无法与外部网络通信。虚拟机可以在宿主机内部创建一个独立的虚拟网络，虚拟机和宿主机可以通过这个虚拟网络进行通信，但虚拟机无法访问宿主机所在的外部网络，也无法被外部网络访问。

六、收获感悟

在本次实验中，通过模拟“红蓝”对抗的方式，从攻击者和防御者的视角对系统进行了设计和测试，使我对构建安全系统有了更深刻的认识。

首先，作为“蓝队”一方，我在系统设计中引入了时间戳、随机数、RSA 签名等多层安全机制来抵御重放攻击、伪造攻击和未授权访问。这些防护措施确保了系统的指令唯一性、真实性和完整性。然而，当我站在“红队”视角分析系统的潜在弱点时，我发现了系统在抗拒绝服务攻击方面的不足，并意识到传输层加密（如 SSL/TLS）在防御中间人攻击中的重要性。

通过这种对抗模拟，我深刻体会到“红蓝”对抗在构建安全系统中的重要性。仅凭单一视角很难覆盖所有潜在威胁，而“红蓝”对抗的过程促使我不断从攻击和防御的角度完善系统设计。对于构建更为稳固的安全系统，红蓝对抗提供了一种更全面、更有效的安全保障思路。